

Operations on Data Structures

Q. Discuss the various fundamental operations that can be performed on data structures, explaining each with a suitable example.

📌 Definition to Remember

Operations on Data Structures are the fundamental actions performed to manipulate, store, and retrieve data. The six core operations are: **Traversal, Insertion, Deletion, Searching, Sorting, and Merging**. The choice of data structure directly determines the efficiency (time and space complexity) of these operations.

The Six Fundamental Operations

Operation	Definition	Example
Traversal	Visiting every element exactly once to process it	Printing all elements of an array
Insertion	Adding a new element to the data structure	Pushing a value onto a Stack
Deletion	Removing an existing element	Popping from a Stack, dequeuing from a Queue
Searching	Finding an element by its key value	Linear search, Binary search on an array
Sorting	Arranging elements in a specific order (asc/desc)	Bubble Sort, Quick Sort on an array
Merging	Combining two or more structures into one	Merging two sorted arrays into one sorted array

Traversal Algorithm Example (Array)

```
Algorithm Traverse(A, N):
1. Set i = 0
2. Repeat while i < N:
3.   Print A[i]
4.   i = i + 1
5. Stop
```

For $A = [10, 20, 30]$, output: $10 \rightarrow 20 \rightarrow 30$

Trade-offs Between Data Structures

DS	Fast Operations	Slow Operations
Array	Searching (index-based, $O(1)$)	Insertion, Deletion (shifting, $O(n)$)
Linked List	Insertion, Deletion ($O(1)$ at head)	Searching ($O(n)$)
Stack/Queue	Insert/Delete at ends ($O(1)$)	Searching middle elements ($O(n)$)

★ **Must-Write Points (for 10 marks)**

1. Six fundamental operations: **Traversal, Insertion, Deletion, Searching, Sorting, Merging**.
2. **Traversal**: visits each element exactly once — used for printing, processing.
3. **Insertion**: adds a new element; location depends on the DS type.
4. **Deletion**: removes an element; memory management of vacated space is required.
5. **Searching**: finds a specific element using its key (linear or binary search).
6. **Sorting**: arranges elements in order — ascending or descending.
7. **Merging**: combines two DS into one (e.g., merging two sorted arrays).

⚡ **Quick Recall**

Traversal (visit all) → Insertion (add) → Deletion (remove) → Searching (find by key) → Sorting (arrange in order) → Merging (combine structures)



1. Array Operations: Traversal, Insertion & Deletion (C Code)

```
#include <stdio.h>

void traverse(int arr[], int n) {
    printf("Array Elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int insertElement(int arr[], int n, int capacity, int element, int pos) {
    if (n >= capacity) {
        printf("Error: Array Overflow! Cannot insert.\n");
        return n;
    }
    for (int i = n - 1; i >= pos; i--) {
        arr[i + 1] = arr[i]; // Shift elements right
    }
    arr[pos] = element;
    return n + 1;
}

int deleteElement(int arr[], int n, int pos) {
    if (pos < 0 || pos >= n) {
        printf("Error: Invalid Position! Cannot delete.\n");
        return n;
    }
    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1]; // Shift elements left
    }
    return n - 1;
}

int main() {
    int arr[10] = {10, 20, 30, 40, 50};
    int n = 5;

    printf("--- Initial Array ---\n");
    traverse(arr, n);

    printf("\n--- Inserting 25 at Index 2 ---\n");
    n = insertElement(arr, n, 10, 25, 2);
    traverse(arr, n);

    printf("\n--- Deleting Element at Index 3 (30) ---\n");
    n = deleteElement(arr, n, 3);
    traverse(arr, n);

    return 0;
}
```

2. Searching Algorithms: Linear Search vs. Binary Search (C Code)

```
#include <stdio.h>

// Linear Search - O(n)
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) return i;
    }
    return -1;
}

// Binary Search - O(log n) [Requires Sorted Array]
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int arr[] = {12, 24, 36, 48, 60, 72, 84};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 48;

    int linRes = linearSearch(arr, n, target);
    printf("Linear Search: Target %d found at Index %d\n", target, linRes);

    int binRes = binarySearch(arr, n, target);
    printf("Binary Search: Target %d found at Index %d\n", target, binRes);

    return 0;
}
```

3. Sorting & Merging Operations (C Code)

```
#include <stdio.h>

// Bubble Sort Operation -  $O(n^2)$ 
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Merging Two Sorted Arrays -  $O(m + n)$ 
void mergeArrays(int A[], int m, int B[], int n, int C[]) {
    int i = 0, j = 0, k = 0;
    while (i < m && j < n) {
        if (A[i] <= B[j]) C[k++] = A[i++];
        else C[k++] = B[j++];
    }
    while (i < m) C[k++] = A[i++];
    while (j < n) C[k++] = B[j++];
}

int main() {
    int A[] = {10, 30, 50};
    int B[] = {20, 40, 60};
    int m = 3, n = 3;
    int C[6];

    mergeArrays(A, m, B, n, C);
    printf("Merged Sorted Array: ");
    for (int i = 0; i < m + n; i++) printf("%d ", C[i]);
    printf("\n");

    return 0;
}
```